# Mapa do ISEP: Aplicação Android para Orientação no Campus

Afonso Teixeira (1250717) e Marco Cardoso (1251212)

Instituto Superior de Engenharia do Porto (ISEP), Licenciatura em Engenharia de Telecomunicações e Informática, Desenvolvimento de Software e Sistemas Móveis (DSSMV), 2026/2027

**Resumo.** Este relatório descreve o Mapa do ISEP, uma aplicação Android em Java que ajuda os alunos do ISEP a orientarem-se no campus. A aplicação autentica o aluno no Firebase Authentication e obtém do Cloud Firestore, com pedidos HTTP construídos na própria aplicação, o horário do dia da turma, os locais do campus e a lista de amigos. No Google Maps mostra a biblioteca, a sala da próxima aula e as posições do aluno e dos amigos. A ocupação da biblioteca é estimada a partir da deteção, pelo telemóvel de cada aluno, dos routers Wi-Fi da biblioteca. O relatório apresenta o contexto, a análise, o design, a implementação, os testes e as conclusões.

**Palavras-chave:** Android, Firebase, REST, Google Maps, GPS, Wi-Fi.

## 1 Contexto e problema

Esta secção apresenta o problema que a aplicação resolve, o objetivo do projeto e as condições impostas pelo enunciado.

### 1.1 Problema e motivação

A localização de salas e serviços num campus pode apresentar dificuldades para os alunos, sobretudo quando ainda não conhecem os edifícios e a organização dos espaços. Para chegar a uma aula, o aluno precisa de consultar o horário, identificar a sala atribuída e perceber onde esta se encontra.

Além da orientação no campus, o projeto pretende facilitar a consulta da disponibilidade de amigos e fornecer uma estimativa da ocupação da biblioteca.

### 1.2 Objetivo do projeto

Desenvolver uma aplicação Android que reúna o horário da turma do aluno e um mapa dos espaços relevantes do ISEP. O aluno pode consultar as aulas do dia, localizar a sala da próxima aula e a biblioteca, gerir amizades, consultar quando terminam as aulas de um amigo e ver uma estimativa da ocupação da biblioteca.

### 1.3 Condições do projeto

O enunciado ProjectDroid [1] exige uma aplicação Android desenvolvida em Java, que comunique com o exterior e explore funcionalidades do dispositivo móvel, como o GPS, sensores ou interfaces de comunicação.

Neste projeto, a comunicação com o exterior é feita com os serviços REST do Firebase (Authentication e Cloud Firestore) [5, 6], através de pedidos HTTP construídos na própria aplicação. As funcionalidades do dispositivo exploradas são a localização (GPS e rede), usada para mostrar a posição do aluno e dos amigos no mapa do campus, e o Wi-Fi, usado para detetar a presença do aluno na biblioteca e estimar a ocupação.

## 2 Análise

Esta secção descreve o que o aluno pretende fazer, como interage com a aplicação e que regras a solução deve respeitar. As escolhas de tecnologia ficam na secção 3. A análise segue o modelo do relatório técnico da unidade curricular [2].

### 2.1 Identificação dos atores

**Ator principal — Aluno:** utilizador com sessão iniciada que consulta o horário da sua turma, se orienta no mapa do ISEP, gere a lista de amigos e consulta a ocupação da biblioteca. Os serviços externos (Firebase e Google Maps) não são atores do utilizador: aparecem na arquitetura (secção 3).

### 2.2 User stories e critérios de aceitação

As histórias exprimem necessidades do aluno; os critérios de aceitação (CA) indicam resultados observáveis para verificar cada uma. Cada história corresponde ao caso de uso com o mesmo número (secção 2.5). Iniciar sessão não é uma história: é um pré-requisito de todas (secção 2.5.2).

**US01 — Consultar o horário**
Como aluno, quero consultar o horário do dia da minha turma, para saber quais são as aulas, a que horas decorrem e em que salas.
– CA01.1 — O aluno consulta apenas o horário da turma definida na sua conta.
– CA01.2 — Cada aula apresenta nome, hora de início, hora de fim e sala.

**US02 — Consultar o mapa**
Como aluno, quero consultar um mapa do ISEP com a minha localização, a sala da próxima aula, a biblioteca e a localização dos meus amigos, para me orientar no campus.

– CA02.1 — O mapa apresenta a biblioteca e a sala da aula que está a decorrer ou, se não houver, da próxima aula do dia.
– CA02.2 — A localização do próprio aluno só é apresentada após autorização.
– CA02.3 — A posição de um aluno é partilhada com os seus amigos enquanto ele tem a página Mapa aberta e a localização autorizada; não há permissões de partilha por amigo. Uma posição com mais de 5 minutos aparece a cinzento, com a indicação de há quanto tempo foi vista (por exemplo "visto há 7 min").

**US03 — Consultar a ocupação da biblioteca**
Como aluno, quero consultar uma estimativa do número de pessoas na biblioteca, para decidir se quero ir até lá.
– CA03.1 — A aplicação apresenta a estimativa mais recente, quando disponível.
– CA03.2 — Se a estimativa não estiver disponível, a aplicação não apresenta um valor como se fosse atual.
– CA03.3 — Um aluno presente só conta uma vez.

**US04 — Consultar a lista de amigos**
Como aluno, quero consultar a minha lista de amigos, para encontrar o amigo cuja informação pretendo consultar.
– CA04.1 — O aluno vê os amigos associados à sua conta, com nome, número e fotografia.

**US05 — Consultar quando um amigo termina as aulas**
Como aluno, quero saber quando um amigo termina as aulas do dia, para combinar um encontro.
– CA05.1 — É apresentado o horário do dia do amigo; a hora a que termina é a hora de fim da última aula.
– CA05.2 — Sem aulas, a aplicação indica-o.

**US06 — Adicionar um amigo**
Como aluno, quero adicionar outro aluno à minha lista de amigos, para poder consultar a informação que partilha comigo.
– CA06.1 — A amizade fica registada nos dois sentidos.
– CA06.2 — A aplicação não cria a mesma amizade em duplicado.

**US07 — Remover um amigo**
Como aluno, quero remover um aluno da minha lista de amigos, para deixar de manter essa relação.
– CA07.1 — Após remover, os dois alunos deixam de constar das listas um do outro e de ver a localização um do outro.

**US08 — Alterar a fotografia de perfil**
Como aluno, quero alterar a minha fotografia de perfil, para os outros alunos me identificarem.
– CA08.1 — Após a alteração, a nova fotografia aparece no perfil e na lista de amigos dos outros alunos.

## 2.3 Modelo de domínio

O modelo de domínio (Fig. 1) representa os conceitos do problema, não a implementação: tem conceitos, associações e multiplicidades, sem operações [8].

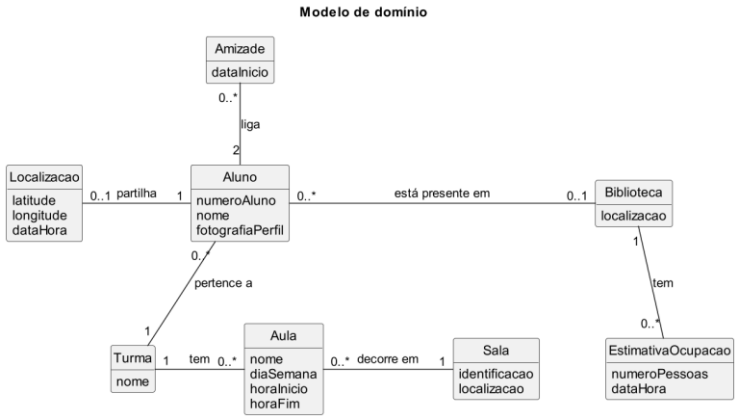



**Fig. 1.** Modelo de domínio

– **Aluno** — identificado pelo número de estudante e pelo nome; pode ter uma fotografia. Pertence a uma única turma.
– **Turma** — agrupamento ao qual o aluno pertence. Uma turma tem várias aulas por semana.
– **Aula** — entrada do horário semanal, com nome, dia da semana, hora de início e de fim. Decorre numa única sala.
– **Sala** — espaço localizável onde decorrem aulas.
– **Amizade** — relação entre exatamente dois alunos; dá acesso recíproco às localizações.
– **Localização** — última posição conhecida de um aluno, com a hora a que foi enviada, partilhada com os amigos (no máximo uma por aluno).
– **Biblioteca** e **EstimativaOcupacao** — a biblioteca tem estimativas de ocupação; cada aluno presente conta uma só vez (CA03.3).

A password não faz parte do modelo: é um dado de autenticação, guardado e validado pelo Firebase Authentication. Assume-se que o horário se repete todas as semanas.

## 2.4 Requisitos não funcionais

Os requisitos não funcionais seguem o modelo FURPS+ (Tabela 1).

**Tabela 1.** Requisitos não funcionais (FURPS+)

| Categoria | Requisito |
|---|---|
| Usabilidade | Interface gráfica Android, com as páginas Início e Mapa. Mensagens claras para credenciais inválidas, falta de ligação, ausência de aulas e sessão expirada. |
| Fiabilidade | Pedidos HTTP com tempo limite de 10 s. Uma falha de rede não termina a aplicação: é apresentada uma mensagem. |
| Desempenho | A comunicação com o servidor corre numa thread secundária, para a interface não bloquear. |
| Suportabilidade | Android, API mínima 26 (Android 8.0). Testes unitários ao modelo, aos mappers, à conversão de JSON e à comunicação HTTP. |
| Segurança | Comunicação só por HTTPS. A password nunca é guardada no dispositivo. A sessão local expira após 2 minutos sem uso. A localização só é usada após autorização. |
| Restrições de implementação | Java; pedidos HTTP com HttpURLConnection e JSON com org.json, sem o SDK do Firebase. |
| Restrições de interface | Firebase Authentication e Cloud Firestore por REST; Google Maps SDK [7]. |

## 2.5 Requisitos funcionais

Os requisitos funcionais são descritos por casos de uso. O diagrama de casos de uso (Fig. 2) mostra o que o aluno pode fazer; a secção 2.5.2 descreve a autenticação, que é o pré-requisito de todos; a Tabela 2 junta as regras que os casos de uso têm de cumprir. Por fim, cada caso de uso tem a sua especificação e o seu diagrama de sequência do sistema (SSD).

### 2.5.1. Diagrama de casos de uso

O ator Aluno tem oito casos de uso (Fig. 2), numerados de UC01 a UC08. Iniciar sessão não aparece como caso de uso: não é um objetivo do aluno, é a condição para usar os outros.

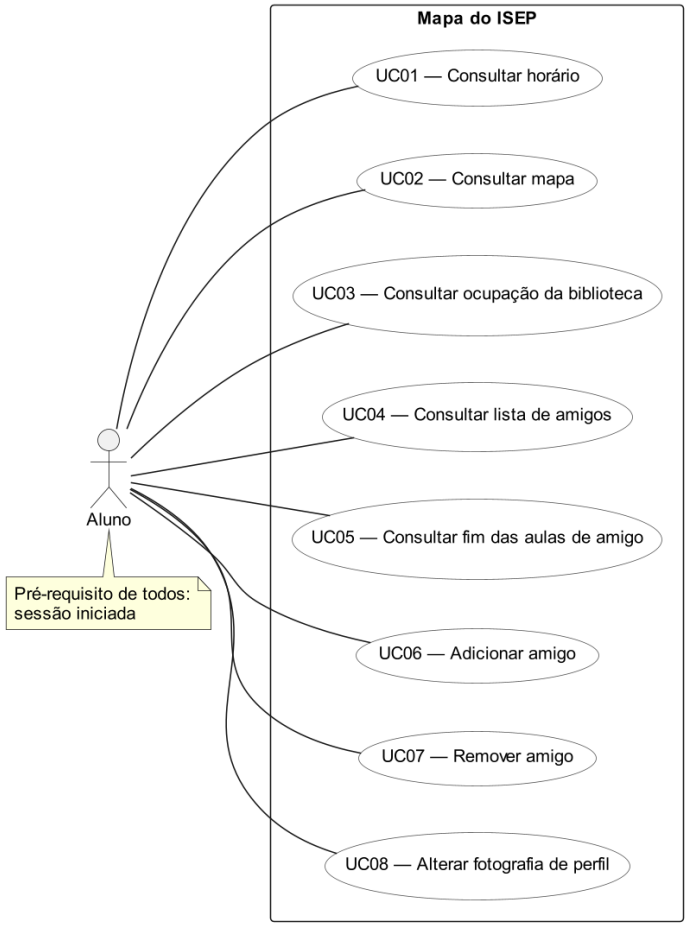



**Fig. 2.** Diagrama de casos de uso

### 2.5.2. Pré-requisito: autenticação

Antes de qualquer caso de uso, o aluno autentica-se com o número de estudante e a password. Aplicam-se as regras seguintes:

– O número tem exatamente 7 dígitos e a password tem entre 1 e 10 caracteres; caso contrário, a aplicação avisa e não envia o pedido.

– Com credenciais inválidas, a aplicação mostra "Utilizador ou password incorretos." e não inicia a sessão.

– Se o aluno sair da aplicação e voltar em menos de 2 minutos, entra sem nova autenticação.

– Em qualquer caso de uso, se a sessão expirar, a aplicação mostra "A sessão terminou. Inicia sessão novamente." e volta ao ecrã de autenticação.

### 2.5.3. Regras funcionais

A Tabela 2 junta as regras que atravessam vários casos de uso.

**Tabela 2.** Regras funcionais

| ID | Regra funcional | Casos de uso |
|---|---|---|
| RF01 | Só alunos registados acedem à aplicação; credenciais inválidas não dão acesso. | Pré-requisito |
| RF02 | A localização do dispositivo só é usada depois de o aluno dar a permissão. | UC02 |
| RF03 | A amizade dá acesso recíproco às localizações; remover a amizade termina esse acesso. | UC02, UC06, UC07 |
| RF04 | Cada aluno presente na biblioteca conta uma só vez na estimativa. | UC03 |

### 2.5.4. Especificação dos casos de uso

Cada caso de uso tem uma tabela com a descrição, as pré e pós-condições e os fluxos básico e alternativos, e um SSD. No SSD o sistema é uma caixa fechada e cada mensagem tem o número do passo do fluxo a que corresponde (por exemplo, 3a é o fluxo alternativo do passo 3) [8].

### UC01 — Consultar horário

A Tabela 3 especifica o caso de uso.

**Tabela 3.** Especificação do UC01 — Consultar horário

| UC01 | Consultar horário |
|---|---|
| **Descrição** | O aluno consulta as aulas do dia da sua turma. |
| **Pré-condição** | Sessão iniciada. |
| **Pós-condição** | As aulas do dia são apresentadas, ou é indicado que não existem. |
| **Fluxo básico** | 1. O aluno abre a página "Início".<br>2. O sistema identifica a turma do aluno e o dia atual.<br>3. O sistema apresenta as aulas do dia, com nome, horas e sala. |
| **Fluxo alternativo** | 3a. Não há aulas hoje: o sistema mostra "Não tens aulas marcadas para hoje." |

A Fig. 3 mostra o SSD do UC01. O sistema só responde depois de identificar a turma e o dia (passo 2).

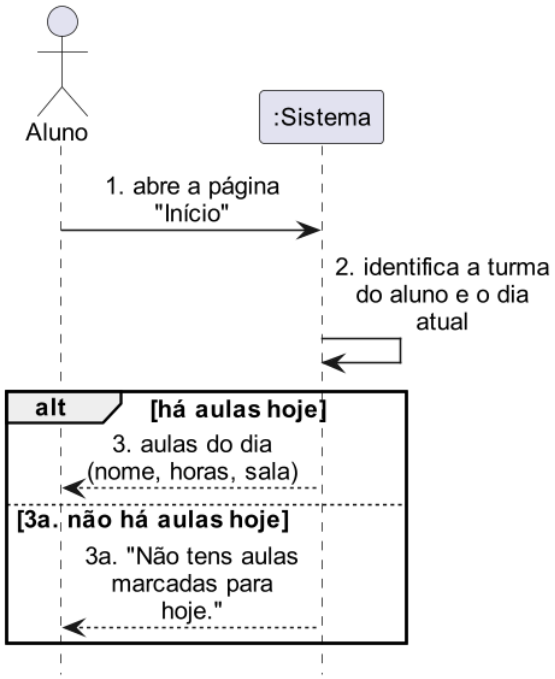



**Fig. 3.** SSD do UC01 — Consultar horário

### UC02 — Consultar mapa

A Tabela 4 especifica o caso de uso.

**Tabela 4.** Especificação do UC02 — Consultar mapa

| **UC02** | **Consultar mapa** |
|---|---|
| **Descrição** | O aluno consulta o mapa do ISEP com a sua posição, a sala da próxima aula, a biblioteca e os amigos. |
| **Pré-condição** | Sessão iniciada. |
| **Pós-condição** | O mapa é apresentado com as posições disponíveis. |
| **Fluxo básico** | 1. O aluno abre a página "Mapa".<br>2. O sistema apresenta o mapa do campus e a biblioteca.<br>3. O sistema assinala a sala da aula atual ou da próxima.<br>4. O sistema apresenta a posição do aluno e a dos amigos. |
| **Fluxo alternativo** | 4a. Localização não autorizada: o sistema pede autorização; se o aluno recusar, o mapa fica sem a sua posição. |

A Fig. 4 mostra o SSD do UC02. O mapa, a biblioteca e a sala (passos 2 e 3) aparecem sem o aluno pedir; a posição depende da autorização (4 ou 4a).

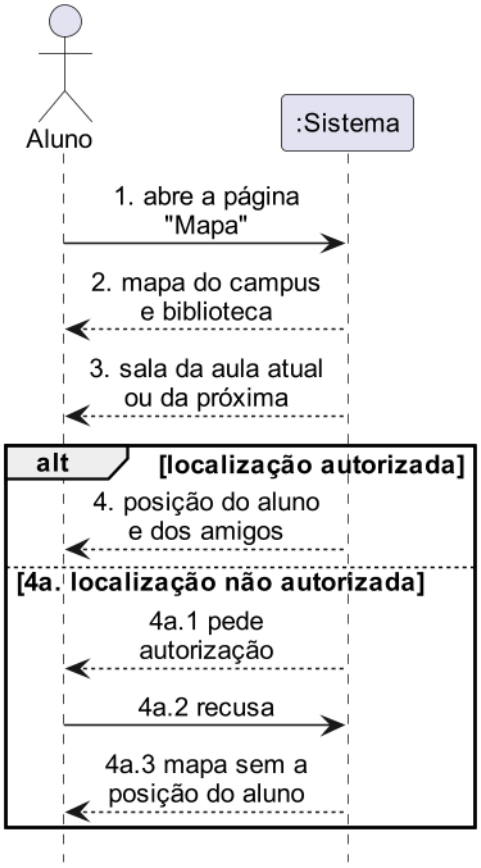



**Fig. 4.** SSD do UC02 — Consultar mapa

### UC03 — Consultar ocupação da biblioteca

A Tabela 5 especifica o caso de uso.

**Tabela 5.** Especificação do UC03 — Consultar ocupação da biblioteca

| **UC03** | **Consultar ocupação da biblioteca** |
|---|---|
| **Descrição** | O aluno consulta uma estimativa do número de pessoas na biblioteca. |
| **Pré-condição** | Sessão iniciada. |
| **Pós-condição** | É apresentada a estimativa, ou a indicação de indisponibilidade. |
| **Fluxo básico** | 1. O aluno abre a página "Início".<br>2. O sistema apresenta "N pessoas neste momento". |
| **Fluxo alternativo** | 2a. Sem localização precisa: o sistema mostra o botão "Ativar localização"; depois de ativada, apresenta a estimativa.<br>2b. Estimativa indisponível: o sistema indica-o. |

A Fig. 5 mostra o SSD do UC03. Sem localização precisa, o aluno tem de a ativar antes de ver a estimativa (2a).

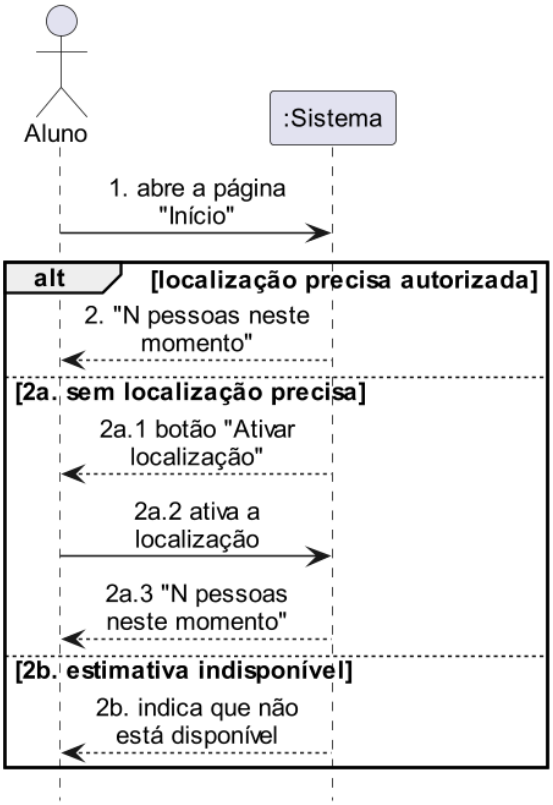



**Fig. 5.** SSD do UC03 — Consultar ocupação da biblioteca

### UC04 — Consultar lista de amigos

A Tabela 6 especifica o caso de uso.

**Tabela 6.** Especificação do UC04 — Consultar lista de amigos

| **UC04** | **Consultar lista de amigos** |
|---|---|
| **Descrição** | O aluno consulta os amigos associados à sua conta. |
| **Pré-condição** | Sessão iniciada. |
| **Pós-condição** | A lista de amigos é apresentada. |
| **Fluxo básico** | 1. O aluno abre "Amigos".<br>2. O sistema apresenta os amigos, com fotografia, nome e número. |
| **Fluxo alternativo** | 2a. Não tem amigos: o sistema mostra "Ainda não adicionaste amigos." |

A Fig. 6 mostra o SSD do UC04. É uma consulta simples: um pedido do aluno e uma resposta do sistema.

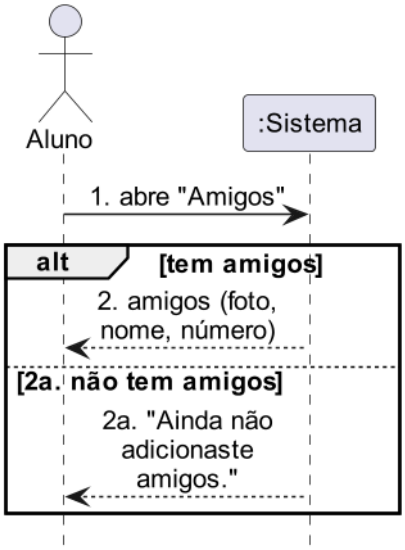



**Fig. 6.** SSD do UC04 — Consultar lista de amigos

### UC05 — Consultar fim das aulas de amigo

A Tabela 7 especifica o caso de uso.

**Tabela 7.** Especificação do UC05 — Consultar fim das aulas de amigo

| **UC05** | **Consultar fim das aulas de amigo** |
|---|---|
| **Descrição** | O aluno consulta a que horas um amigo termina as aulas do dia. |
| **Pré-condição** | Sessão iniciada e amigo na lista. |
| **Pós-condição** | São apresentadas as aulas do dia do amigo. |
| **Fluxo básico** | 1. O aluno carrega em "Ver horário" no cartão de um amigo.<br>2. O sistema apresenta as aulas de hoje do amigo, por ordem; a última indica a hora a que termina. |
| **Fluxo alternativo** | 2a. O amigo não tem aulas hoje: o sistema mostra "Este amigo não tem aulas hoje." |

A Fig. 7 mostra o SSD do UC05. A hora a que o amigo termina é a hora de fim da última aula apresentada.

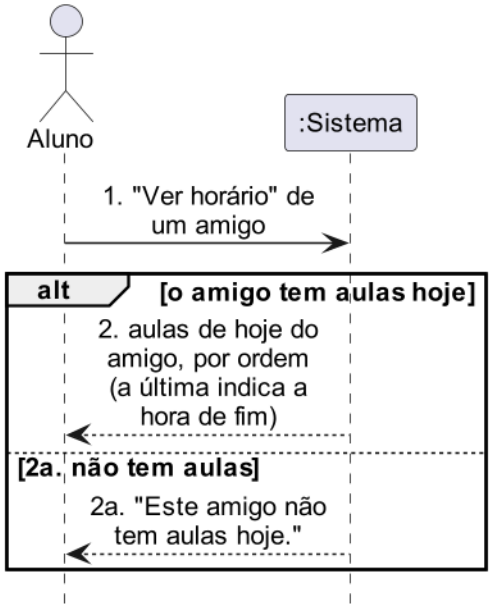



**Fig. 7.** SSD do UC05 — Consultar fim das aulas de amigo

### UC06 — Adicionar amigo

A Tabela 8 especifica o caso de uso.

**Tabela 8.** Especificação do UC06 — Adicionar amigo

| **UC06** | **Adicionar amigo** |
|---|---|
| **Descrição** | O aluno adiciona outro aluno à sua lista de amigos. |
| **Pré-condição** | Sessão iniciada. |
| **Pós-condição** | Os dois alunos passam a constar das listas um do outro. |
| **Fluxo básico** | 1. O aluno escreve o número de estudante de outro aluno.<br>2. O aluno carrega em "Adicionar".<br>3. O sistema valida o número e procura o aluno.<br>4. O sistema regista a amizade nos dois sentidos e mostra "Amigo adicionado." |
| **Fluxo alternativo** | 3a. Número sem 7 dígitos: o sistema indica o erro.<br>3b. Não existe aluno com esse número, ou 3c. é o próprio aluno: o sistema indica o erro e não grava nada.<br>3d. Já é amigo: o sistema mostra "Este aluno já é teu amigo." e não grava nada. |

A Fig. 8 mostra o SSD do UC06. As quatro alternativas (3a a 3d) terminam sem gravar nada.

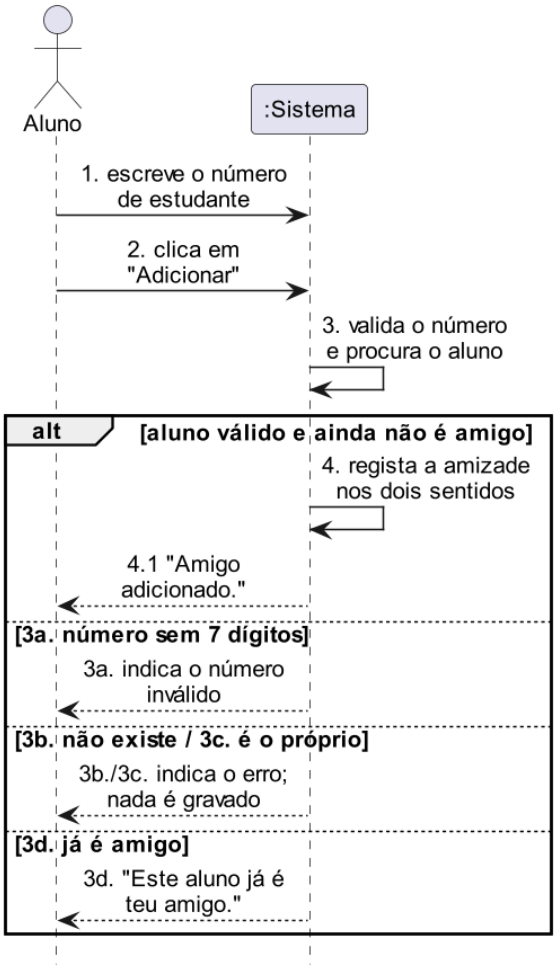



**Fig. 8.** SSD do UC06 — Adicionar amigo

### UC07 — Remover amigo

A Tabela 9 especifica o caso de uso.

**Tabela 9.** Especificação do UC07 — Remover amigo

| **UC07** | **Remover amigo** |
|---|---|
| **Descrição** | O aluno remove um aluno da sua lista de amigos. |
| **Pré-condição** | Sessão iniciada e amigo na lista. |
| **Pós-condição** | A amizade deixa de existir nos dois sentidos. |

| UC07 | Remover amigo |
|---|---|
| **Fluxo básico** | 1. O aluno carrega em "Remover" no cartão do amigo.<br>2. O sistema remove a amizade nos dois sentidos e mostra "Amigo removido." |
| **Fluxo alternativo** | 2a. A remoção falha: o sistema mostra uma mensagem de erro. |

A Fig. 9 mostra o SSD do UC07. A amizade é removida dos dois lados no mesmo passo (2).

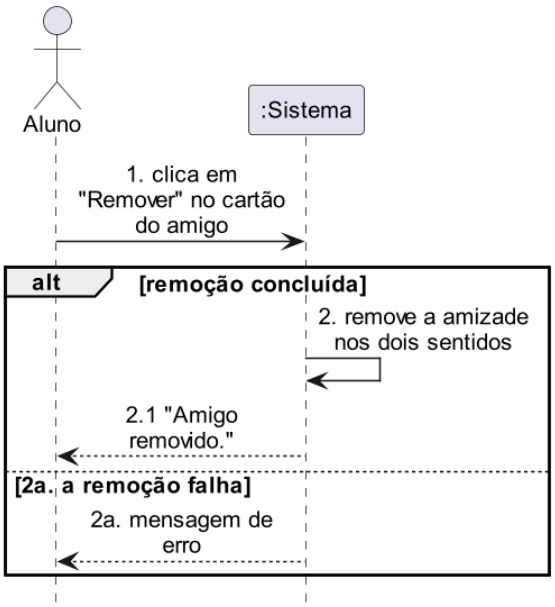



**Fig. 9.** SSD do UC07 — Remover amigo

### UC08 — Alterar fotografia de perfil

A Tabela 10 especifica o caso de uso.

**Tabela 10.** Especificação do UC08 — Alterar fotografia de perfil

| UC08 | Alterar fotografia de perfil |
|---|---|
| **Descrição** | O aluno altera a fotografia do seu perfil. |
| **Pré-condição** | Sessão iniciada. |
| **Pós-condição** | A nova fotografia fica guardada e é apresentada no perfil. |
| **Fluxo básico** | 1. O aluno abre o perfil e carrega em "Alterar foto".<br>2. O aluno escolhe uma imagem.<br>3. O sistema apresenta a nova fotografia e guarda-a. |
| **Fluxo alternativo** | 3a. A gravação falha: o sistema mostra uma mensagem de erro. |

A Fig. 10 mostra o SSD do UC08. A fotografia aparece logo (passo 3); se a gravação falhar, o aluno é avisado (3a).

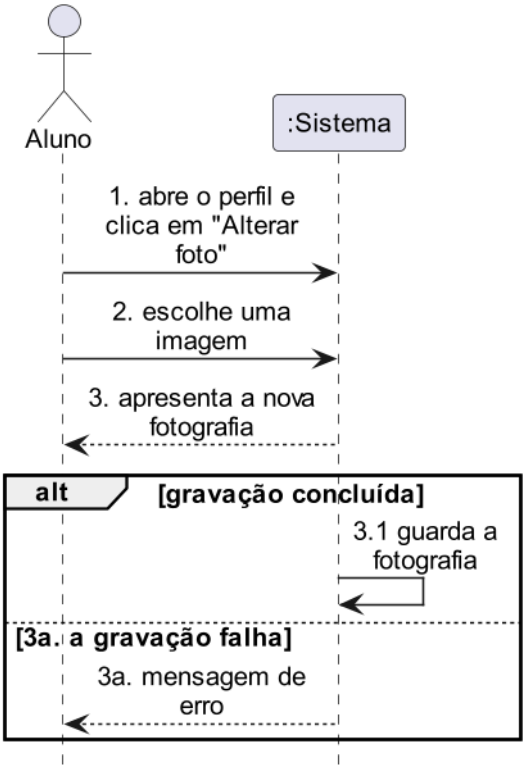



**Fig. 10.** SSD do UC08 — Alterar fotografia de perfil

# 3 Design

Esta secção descreve como a aplicação está construída: a arquitetura física e lógica, a organização do código, as classes, a colaboração entre elas em cada caso de uso e a interface.

## 3.1 Arquitetura física

O diagrama de deployment (Fig. 11) tem dois nós, como o exemplo da unidade curricular [2]: o telemóvel Android, onde o ambiente de execução Android corre o

artefacto MapaISEP.apk e onde estão o GPS e o Wi-Fi, e os servidores da Google, onde correm o Firebase Authentication, o Cloud Firestore e o Google Maps Platform. As linhas entre os nós dizem o protocolo: tudo passa por HTTPS. Este diagrama responde à pergunta "onde corre cada coisa e por onde passa a comunicação"; por isso mostra máquinas, hardware e protocolos, e não as partes do código.

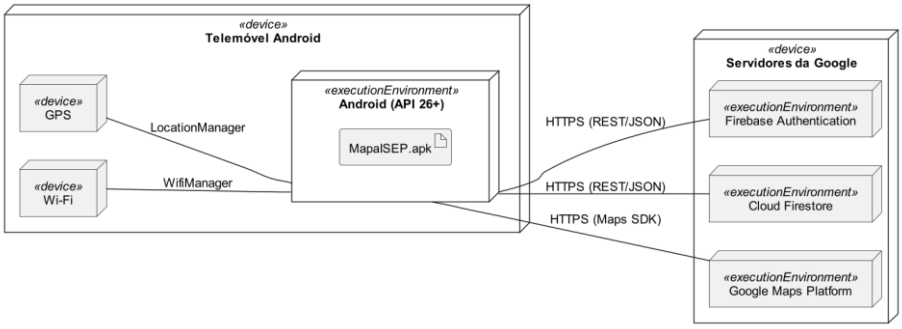



**Fig. 11.** Diagrama de deployment

– **Firebase Authentication** — valida o número de estudante e a password. O número é convertido num email e enviado por POST para o endpoint signInWithPassword [5]. A resposta traz o uid, o idToken e o refreshToken; quando o idToken está perto de expirar (1 hora), a aplicação pede outro ao endpoint securetoken, sem pedir a password.
– **Cloud Firestore** — guarda os alunos (com os amigos e a fotografia), as aulas, os locais do campus, as posições partilhadas (localizacoes) e as presenças na biblioteca (presencas) [6]. Cada pedido leva o cabeçalho Authorization: Bearer <idToken>, para as regras de segurança identificarem o aluno.
– **Google Maps Platform** — fornece o mapa, através do Maps SDK [7].
– **GPS** e **Wi-Fi** — hardware do telemóvel, lidos com o LocationManager [3] e o WifiManager [4] do Android, só depois de o aluno autorizar a localização.

### 3.2 Arquitetura lógica

A Fig. 12 mostra os componentes da aplicação e como dependem uns dos outros. Cada ligação usa a notação "ball-and-socket": a bolinha é a interface que um componente oferece e a meia-lua é a interface de que outro precisa. Os componentes da aplicação estão dentro da caixa App Android. Os sistemas externos são os mesmos da Fig. 11, mas aqui aparecem como componentes que oferecem uma interface, e não como máquinas: o Firebase oferece duas interfaces REST (Authentication e Cloud Firestore), de que o Handler precisa, e o Google Maps Platform oferece a Maps API, de que o Maps SDK precisa. O GPS e o Wi-Fi não aparecem como hardware: o código só os vê através do componente Android (LocationManager e WifiManager).

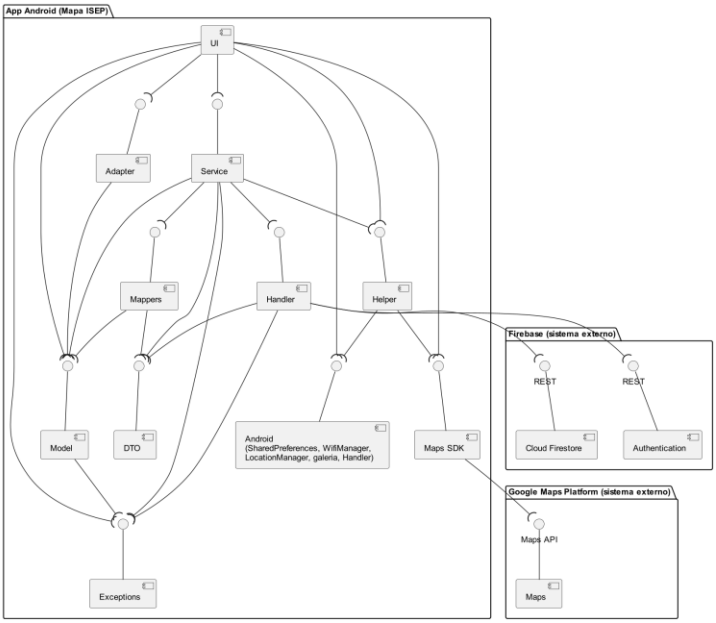



**Fig. 12.** Diagrama de componentes

– **UI** — ecrãs e partes do ecrã; usa o Adapter para mostrar listas, o Service para obter dados, o Maps SDK para desenhar o mapa e o Android para o GPS (LocationManager) e para escolher a fotografia na galeria.
– **Service** — a lógica de cada funcionalidade; usa o Handler para falar com o Firebase, os Mappers para obter objetos do modelo e o Helper para coisas do Android.
– **Handler** — envia os pedidos HTTP e converte JSON em DTO e DTO em JSON; é o único componente que fala com o Firebase.
– **Mappers**, **DTO**, **Model** e **Exceptions** — o caminho dos dados: o DTO tem os dados como vêm da API, o mapper converte-os no modelo e o modelo valida-os, lançando exceções se forem inválidos.
– **Helper** — guarda a sessão (SharedPreferences), lê as redes Wi-Fi (WifiManager) e cria o ícone redondo da fotografia para o mapa (Maps SDK).
– **Android** e **Maps SDK** — bibliotecas que correm dentro da aplicação: o Android dá acesso ao telemóvel e o Maps SDK desenha o mapa, pedindo os dados ao Google Maps Platform.

### 3.3 Organização do código

O código está no pacote pt.isep.dssmv.mapaisep e segue a organização do exemplo da unidade curricular (School Disk Client) [2]: a porta de entrada (LoginActivity) fica na raiz e o resto em pacotes (Fig. 13).

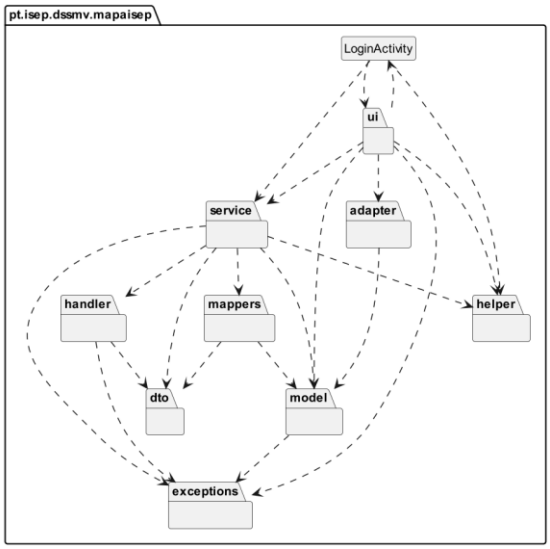



**Fig. 13.** Diagrama de pacotes

As setas da Fig. 13 são dependências tiradas dos imports do código. Quase todas apontam para baixo; a exceção é a seta da ui para a LoginActivity, porque ao terminar a sessão (ou quando a sessão expira) a aplicação volta ao ecrã de login. A Tabela 11 diz o que cada pacote faz.

**Tabela 11.** Responsabilidade de cada pacote

| **Pacote** | **Responsabilidade** |
|---|---|
| ui | Ecrãs (Activities) e partes do ecrã: recebem as ações do aluno e chamam os services. |
| adapter | Liga a lista de amigos (List<Amigo>) à ListView do ecrã. |
| service | Lógica de cada funcionalidade: autenticação, horário, amigos, perfil, locais, localizações, biblioteca e presença. |
| handler | Pedidos HTTP (NetworkHandler, HttpResponse) e conversão JSON ↔ DTO (JsonHandler). |
| helper | Ferramentas do Android e constantes: sessão (SharedPreferences), Wi-Fi, ícone da fotografia, endereços e parâmetros. |
| dto | Dados no formato da API, sem lógica. |
| mappers | Conversão DTO ↔ modelo. |
| model | Aula, Amigo e Local, que validam os seus dados. |
| exceptions | InvalidDataException, DuplicatedDataException e SessaoExpiradaException. |

### 3.4 Diagramas de classes

Há um diagrama de classes por pacote. Só aparecem os métodos importantes para perceber o papel de cada classe.

**Pacote model**
A Fig. 14 mostra o modelo. Segue-se o princípio Information Expert do GRASP [8]: quem tem a informação é quem a valida. A Aula, o Amigo e o Local verificam os seus dados nos setters e lançam InvalidDataException se forem inválidos (por exemplo, uma aula cuja hora de fim não é posterior à de início, ou um número de estudante sem 7 dígitos). O Amigo guarda também a posição partilhada, que fica vazia enquanto o amigo não a partilhar. A aula liga-se ao local pelo código da sala: o salaId da aula é igual ao id do documento em locais (por exemplo, B201).

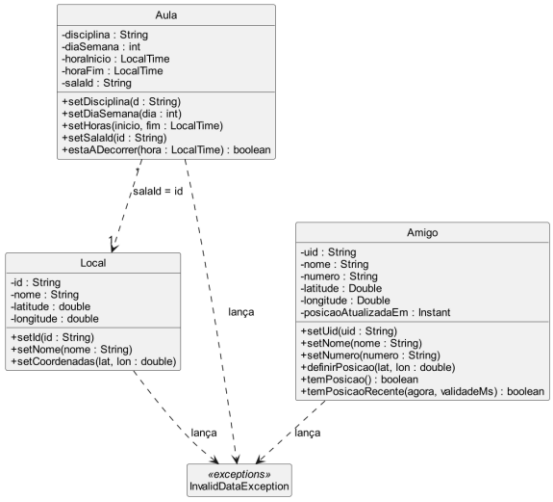



**Fig. 14.** Diagrama de classes — model

**Pacote ui**
Na Fig. 15, as cinco Activities são os ecrãs. O ecrã principal (MenuPrincipalActivity) divide o trabalho por quatro partes: MapaCampus (campus, biblioteca e sala da próxima aula), LocalizacaoAluno (posição do aluno), AmigosNoMapa (posição dos amigos, de 5 em 5 segundos) e CartaoBiblioteca (lotação da biblioteca). O ecrã cria-as, entrega-lhes o mapa e liga-as ou desliga-as no onResume e no onPause. O detetor de presença na biblioteca não pertence a nenhum ecrã: é um só para toda a aplicação (secção 4.6). O MenuConta é o menu ≡ dos ecrãs Amigos, Perfil e Horário do amigo, e também trata da sessão expirada em todos os ecrãs.

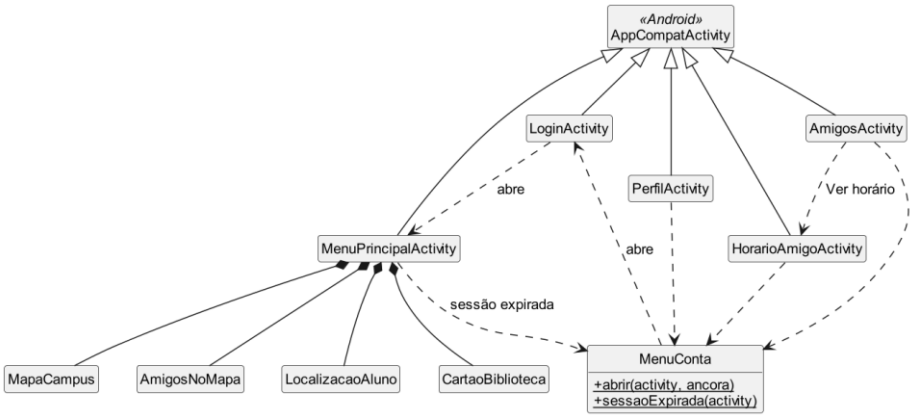



**Fig. 15.** Diagrama de classes — ui

**Pacotes adapter e helper**

Na Fig. 16, o ListViewAdapterAmigos repete o molde item_amigo.xml por cada amigo e preenche a fotografia, o nome e o número; os botões Remover e Ver horário avisam a Activity através da interface AcoesAmigo, porque o adapter não chama serviços. O helper tem as ferramentas usadas em vários sítios: a sessão guardada em SharedPreferences (SessaoHelper), a pesquisa de redes Wi-Fi (WifiHelper), o ícone redondo da fotografia no mapa (IconeFotoHelper) e as constantes.

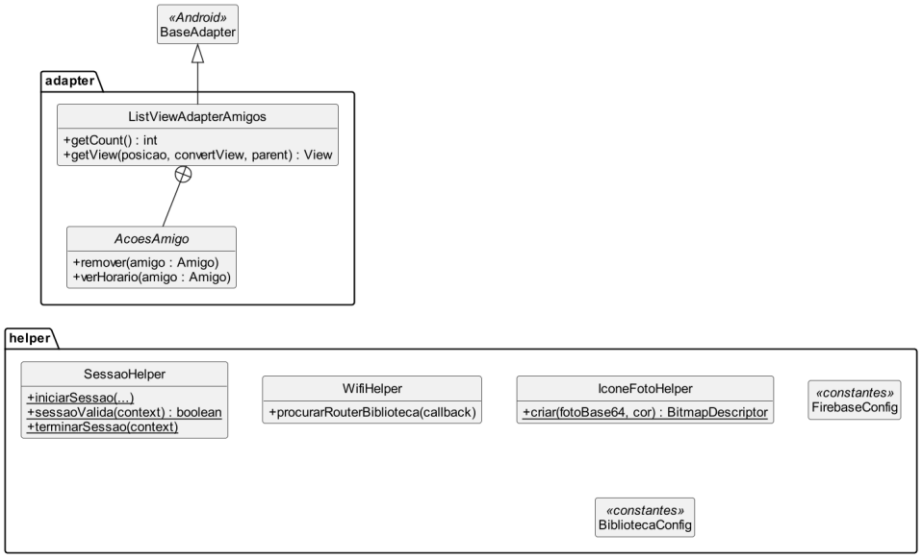



**Fig. 16.** Diagrama de classes — adapter e helper

## Pacote service

A Fig. 17 mostra os services. Cada um define uma interface de callback (por exemplo, HorarioCallback) que o ecrã implementa. Antes de cada pedido, o service pede ao AuthService um idToken válido (obterIdTokenValido). O HorarioService tem também o obterProximaAula, que escolhe a aula atual ou a próxima: é lógica sobre aulas, por isso não está no ecrã do mapa.

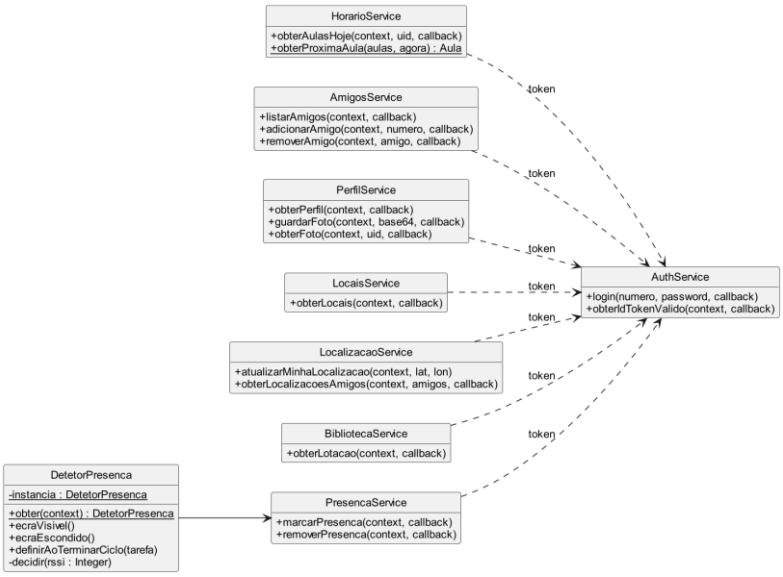



**Fig. 17.** Diagrama de classes — service (atualizada)

**Pacotes handler e exceptions**

Na Fig. 18, o NetworkHandler envia os pedidos e devolve um HttpResponse com o código e o corpo; os services decidem o que fazer com o código (por exemplo, 404 na lista de amigos quer dizer que ainda não há amigos). Um 401 num pedido com token lança SessaoExpiradaException. O JsonHandler é o único que conhece o formato do Firestore e converte JSON em DTO e DTO em JSON.

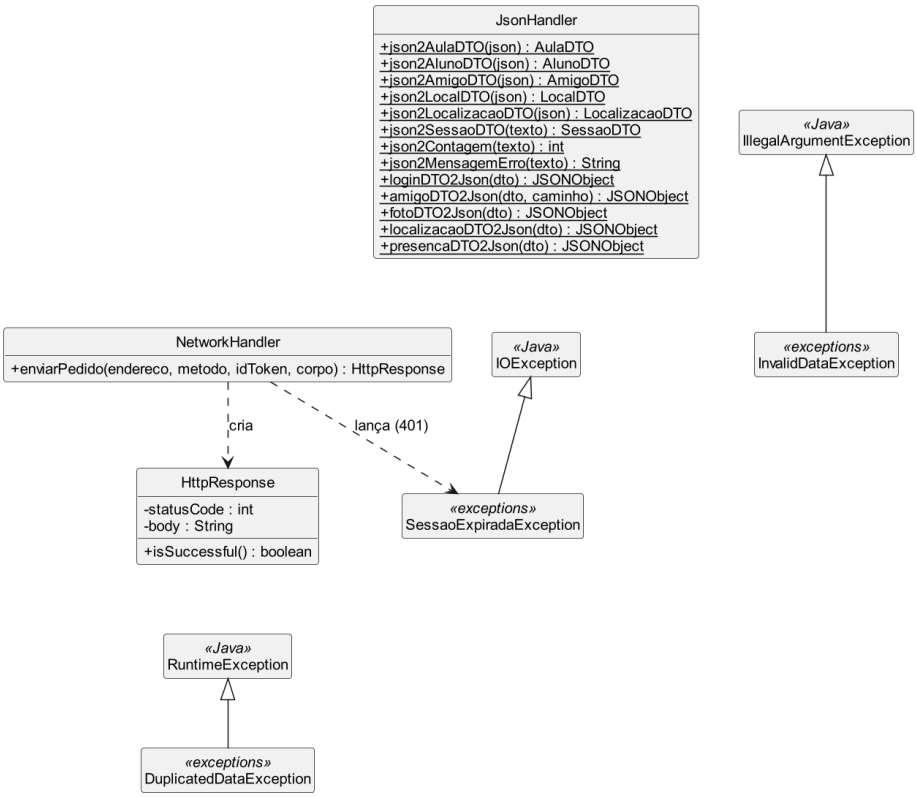



**Fig. 18.** Diagrama de classes — handler e exceptions

### Pacotes dto e mappers

Na Fig. 19, cada tipo de dado tem um DTO, com os dados como vêm da API, e um mapper, que o converte no modelo. Há DTOs sem modelo próprio: o LocalizacaoDTO acrescenta a latitude e a longitude ao Amigo, e o LoginDTO, o SessaoDTO e o PresencaDTO só existem para enviar ou receber dados da autenticação e da biblioteca.

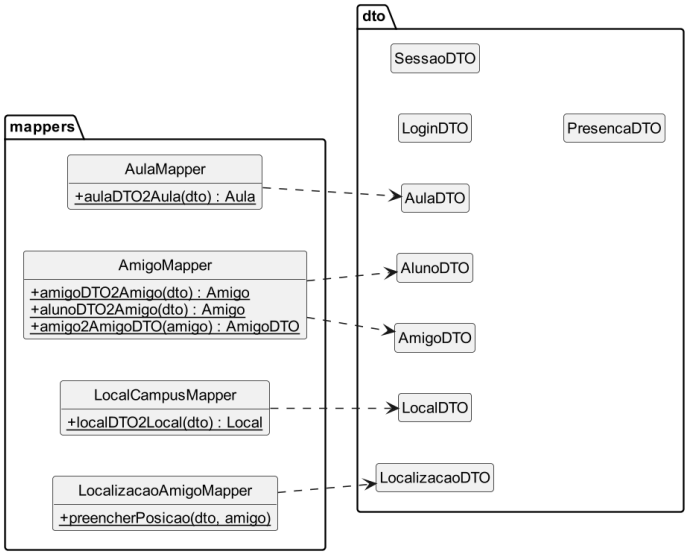



**Fig. 19.** Diagrama de classes — dto e mappers

### 3.5 Diagramas de sequência

Os diagramas de sequência mostram como as classes colaboram em cada caso de uso. Para se lerem, só têm os participantes principais; a thread secundária e o Handler aparecem numa nota.

**Autenticação**
Na Fig. 20, o AuthService converte as credenciais em JSON (LoginDTO), envia o pedido e, se a resposta for 200, converte-a num SessaoDTO; a sessão fica guardada pelo SessaoHelper.

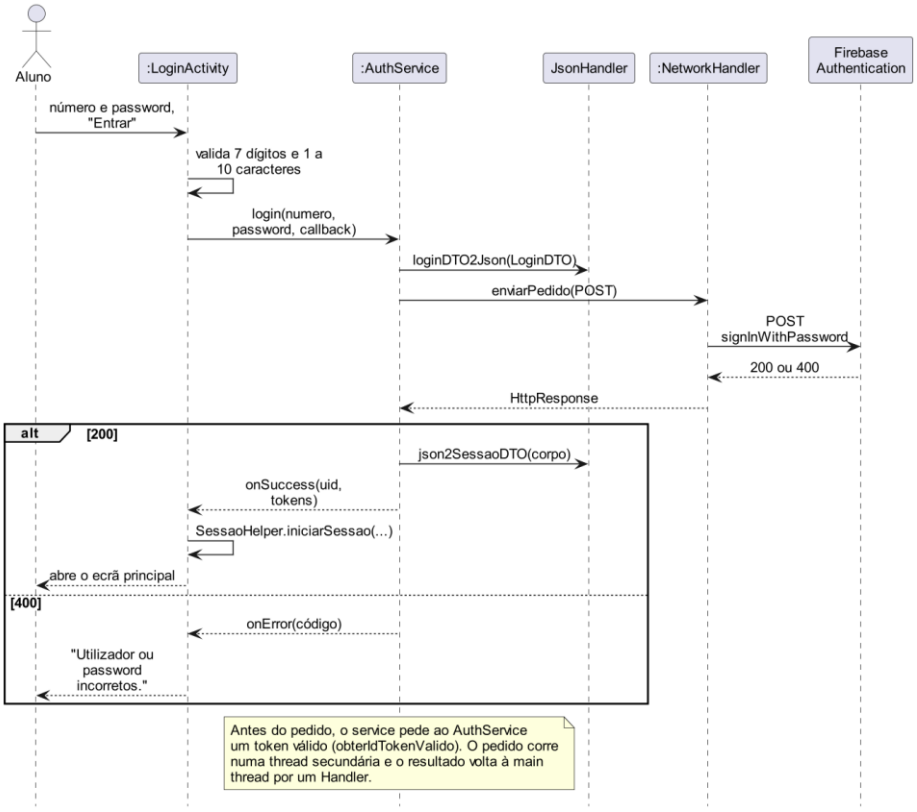



**Fig. 20.** Diagrama de sequência — Autenticação

### UC01 — Consultar horário

Na Fig. 21, o HorarioService faz dois pedidos: o documento do aluno, para saber a turma, e a pesquisa das aulas dessa turma no dia. Cada aula segue o caminho JSON → AulaDTO → Aula.

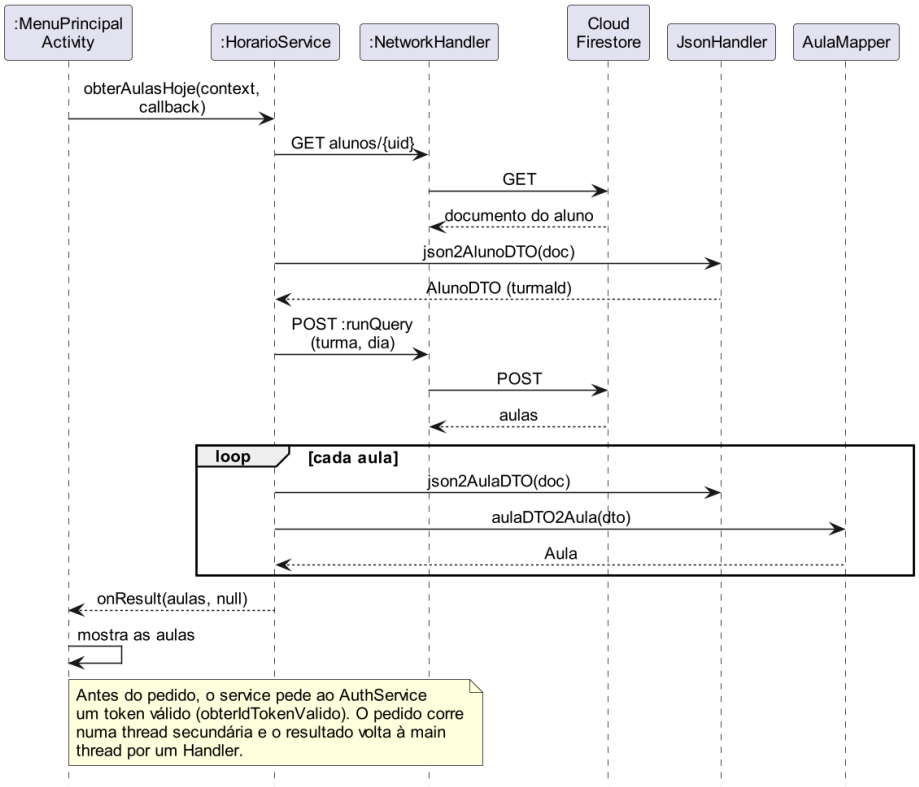



**Fig. 21.** Diagrama de sequência — UC01 — Consultar horário

### UC02 — Consultar mapa

Na Fig. 22, o ecrã principal entrega o mapa às três partes que desenham nele: o MapaCampus (biblioteca e, depois de receber as aulas com definirAulas, a sala da próxima aula), a LocalizacaoAluno (posição do aluno) e o AmigosNoMapa (amigos, de 5 em 5 segundos).

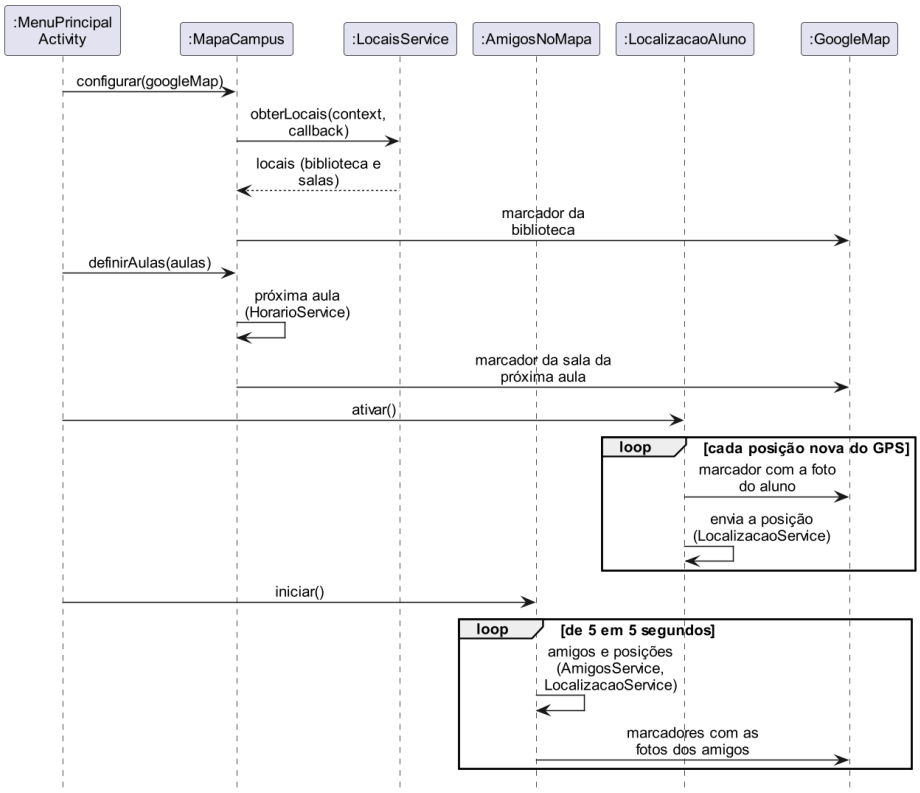



**Fig. 22.** Diagrama de sequência — UC02 — Consultar mapa

### UC03 — Consultar ocupação da biblioteca

Na Fig. 23, a lotação é contada pelo próprio Firestore; ao mesmo tempo, o DetetorPresenca procura o router da biblioteca de 30 em 30 segundos e marca ou retira a presença do aluno. Qualquer ecrã visível liga o detetor (ecraVisivel); no fim de cada ciclo, o detetor avisa o cartão da biblioteca, que volta a pedir a lotação.

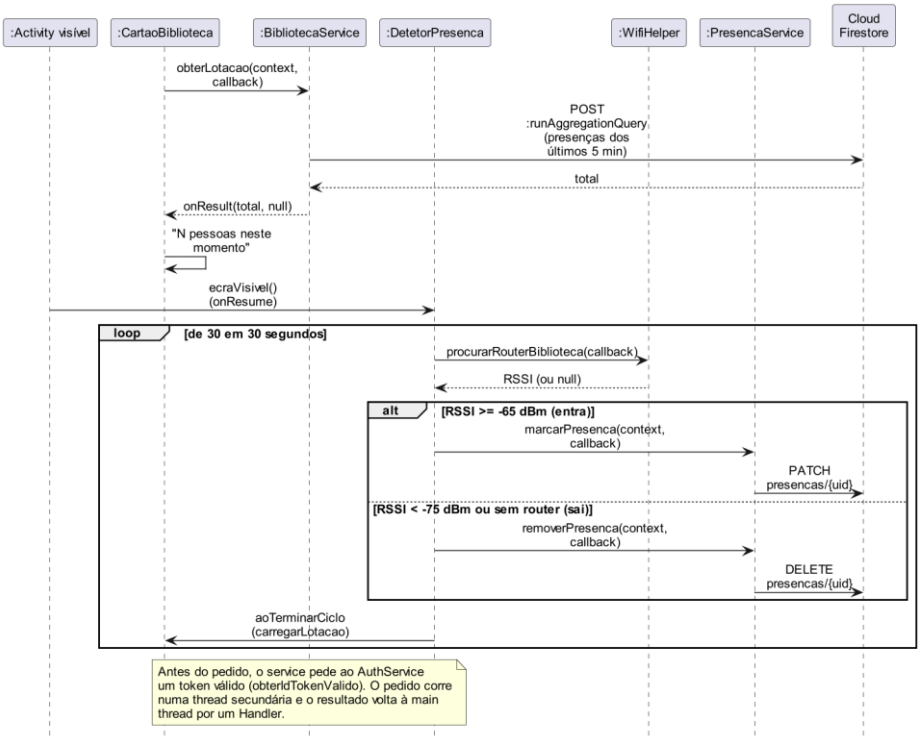



**Fig. 23.** Diagrama de sequência — UC03 — Consultar ocupação da biblioteca (atualizada)

## UC04 — Consultar lista de amigos

Na Fig. 24, os amigos chegam como JSON, passam a AmigoDTO e a Amigo, e o adapter cria um cartão por amigo, como no exemplo da unidade curricular [2].

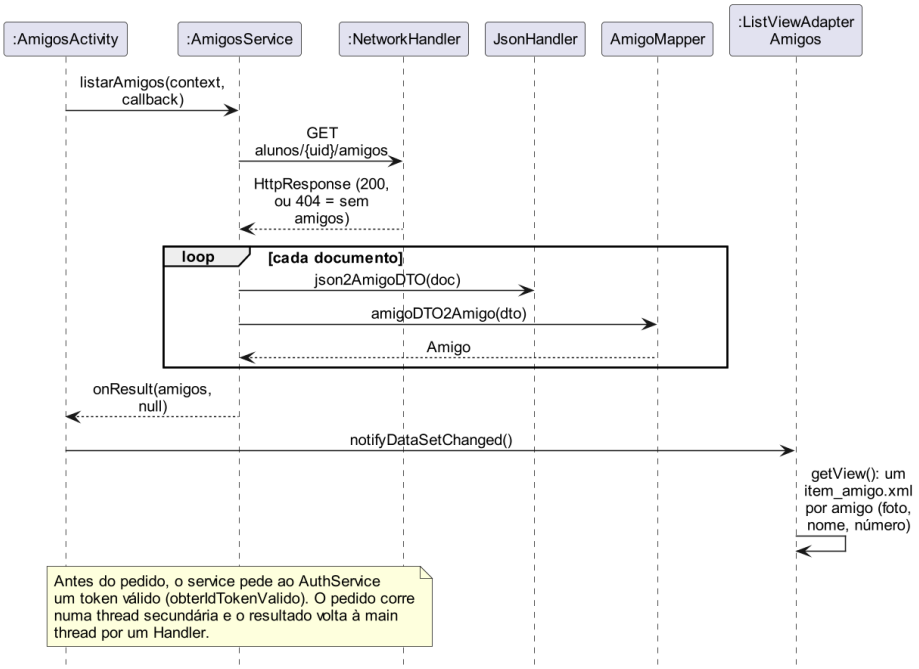



**Fig. 24.** Diagrama de sequência — UC04 — Consultar lista de amigos

### UC05 — Consultar fim das aulas de amigo

Na Fig. 25, o uid do amigo vai no Intent para o ecrã do horário, que usa o mesmo HorarioService do UC01.

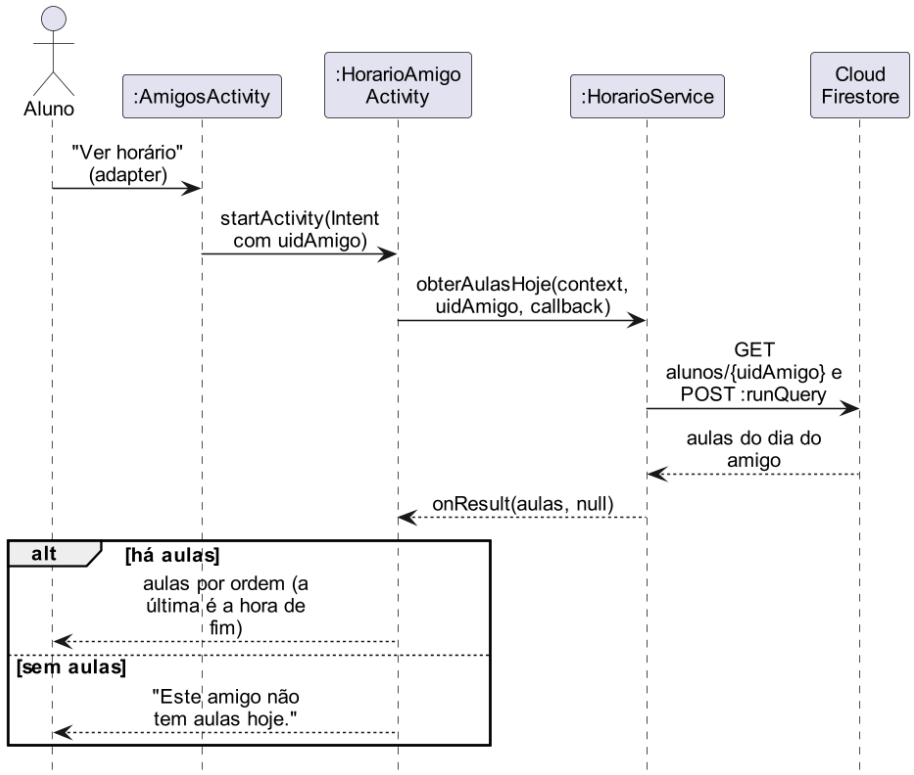



**Fig. 25.** Diagrama de sequência — UC05 — Consultar fim das aulas de amigo

**UC06 — Adicionar amigo**

Na Fig. 26, o service procura o aluno pelo número (tem de existir e não pode ser o próprio) e, antes de gravar, verifica se já é amigo; se for, lança DuplicatedDataException e o ecrã mostra a mensagem. A amizade é gravada nos dois sentidos num só pedido (:commit).

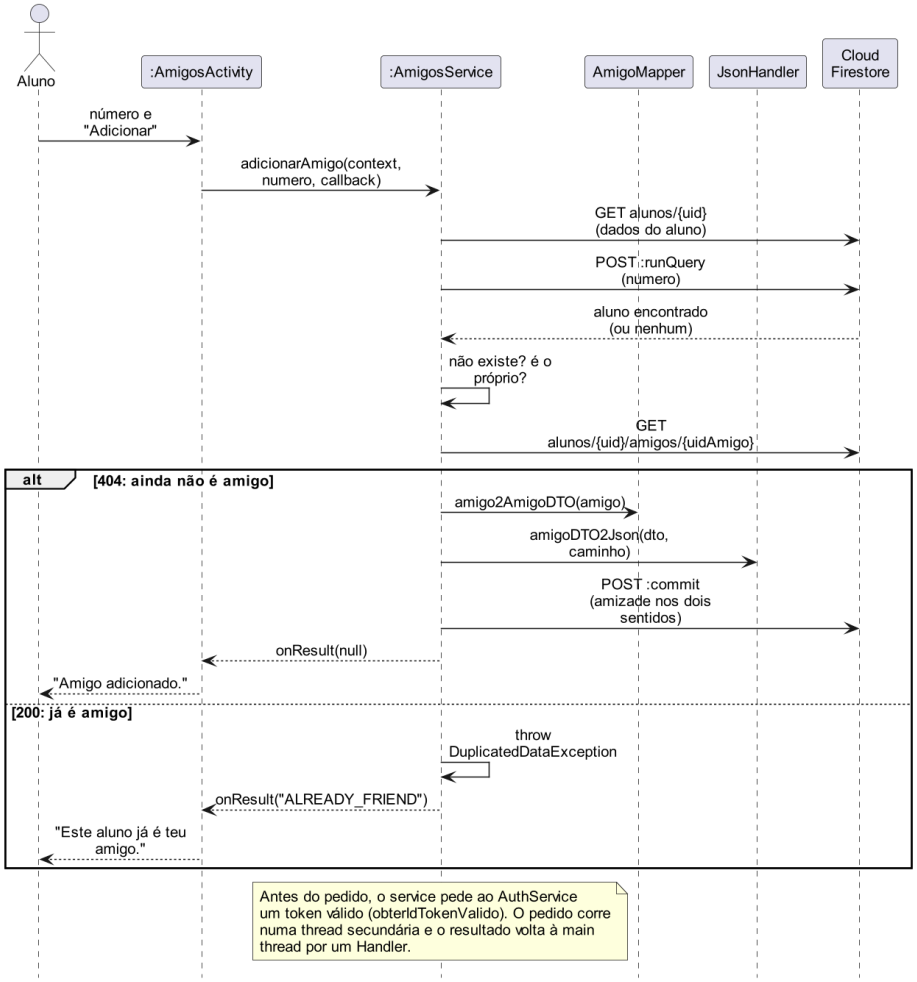



**Fig. 26.** Diagrama de sequência — UC06 — Adicionar amigo

### UC07 — Remover amigo

Na Fig. 27, o botão Remover do cartão (adapter) avisa a Activity, que pede ao service para apagar a amizade nos dois sentidos.

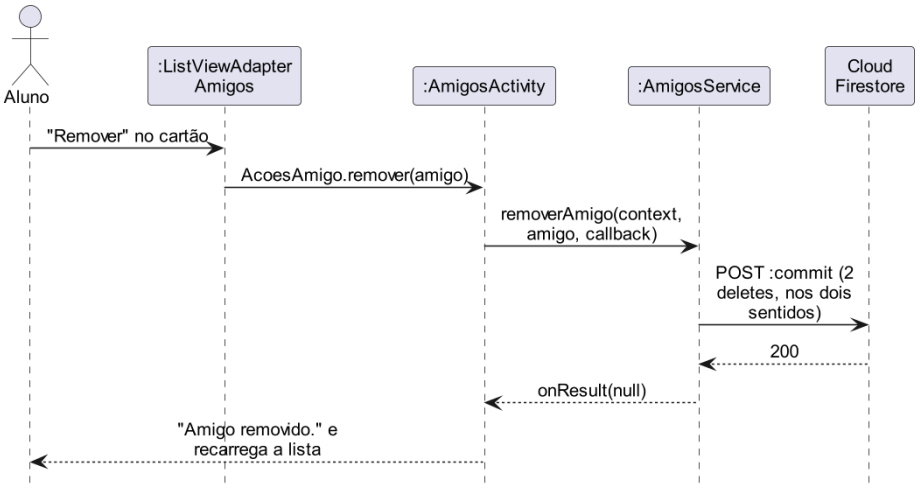



**Fig. 27.** Diagrama de sequência — UC07 — Remover amigo

**UC08 — Alterar fotografia de perfil**

Na Fig. 28, a imagem é reduzida e convertida em Base64 no telemóvel e guardada com um PATCH que só altera o campo fotoBase64.

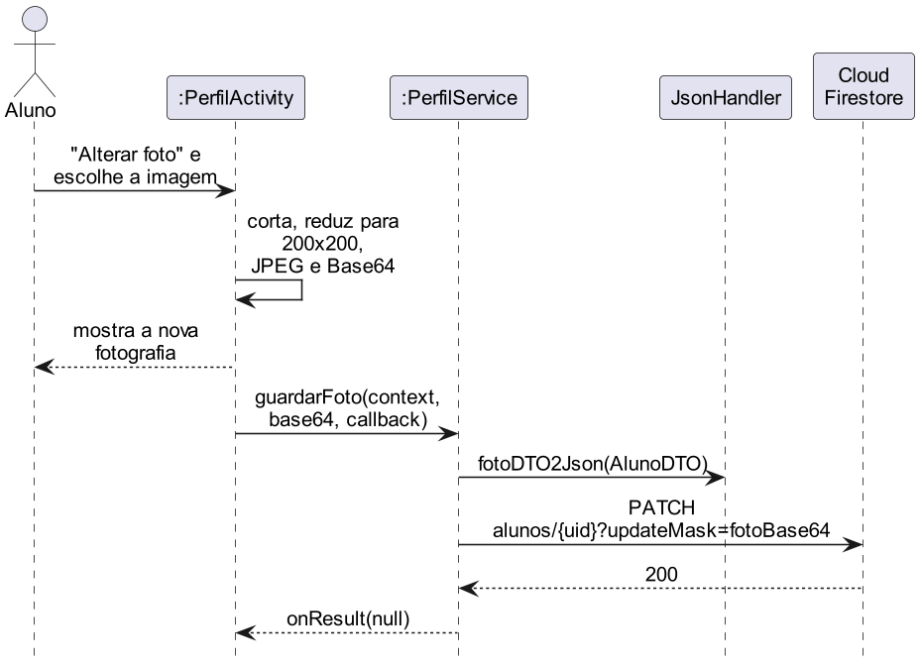



**Fig. 28.** Diagrama de sequência — UC08 — Alterar fotografia de perfil

### 3.6 Interface com o utilizador

Os mockups definem os ecrãs antes da implementação. A Fig. 29 mostra o ecrã de autenticação (a) e o ecrã principal: a página Início com o horário e a biblioteca (b), a página Mapa (c) e o menu de conta ≡ (d). As páginas Início e Mapa trocam com os botões de baixo.

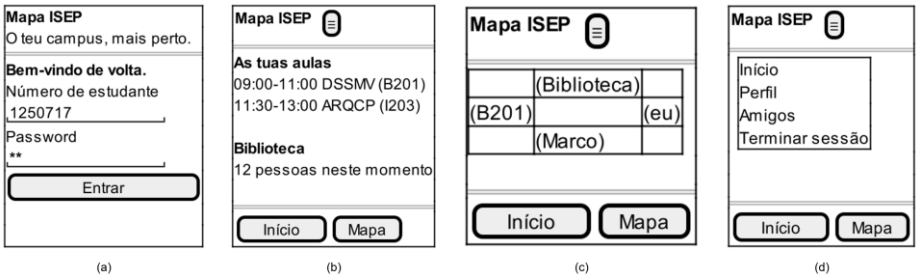



**Fig. 29.** Mockups: (a) autenticação, (b) Início, (c) Mapa, (d) menu de conta

A Fig. 30 mostra os ecrãs que se abrem a partir do menu de conta: Amigos (a), com o campo para adicionar e um cartão por amigo, o horário de um amigo (b), aberto pelo botão Ver horário, e o Perfil (c), com a fotografia e o botão para a alterar.

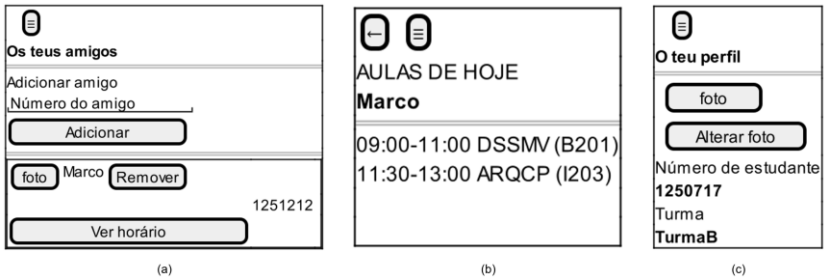



**Fig. 30.** Mockups: (a) Amigos, (b) Horário do amigo, (c) Perfil

### 3.7 Decisões técnicas

– **Sem o SDK do Firebase.** A unidade curricular exige comunicação HTTP explícita, com pelo menos três métodos HTTP. Por isso, os pedidos são construídos à mão com HttpURLConnection e o JSON com org.json, e a aplicação usa quatro métodos: GET, POST, PATCH e DELETE. O HttpURLConnection não aceita PATCH: o pedido segue como POST com o cabeçalho X-HTTP-Method-Override: PATCH.

– **Sessão.** A sessão é guardada em SharedPreferences (SessaoHelper); se o aluno voltar em menos de 2 minutos, entra sem nova autenticação. O idToken dura 1 hora e é renovado automaticamente com o refreshToken.

– **Fotografia em Base64.** A imagem é reduzida para 200×200 píxeis, comprimida em JPEG e guardada como texto no documento do aluno. Assim não é preciso o Firebase Storage.

– **Amizade nos dois sentidos.** Gravar e remover usam um único pedido :commit com duas escritas, que acontecem as duas ou nenhuma.

– **Biblioteca por Wi-Fi.** A presença é detetada pela intensidade do sinal (RSSI) dos routers da biblioteca, com limiares diferentes para entrar e sair (secção 4.6). A contagem é feita pelo Firestore (runAggregationQuery), para cada aluno contar uma só vez. Há um só detetor para toda a aplicação (Singleton), ligado enquanto algum ecrã estiver visível.
– **Localização.** O ouvinte do GPS implementa os quatro métodos do LocationListener, porque no Android 8 a 10 três deles não têm versão por omissão e a sua falta termina a aplicação [3]. A última posição conhecida só é usada se tiver menos de 2 minutos.

# 4 Implementação

O código segue a convenção camel case e tem os comentários em português. Esta secção explica as partes mais importantes; os métodos principais aparecem em listagens com as linhas numeradas, sem os comentários.

## 4.1 Comunicação com o Firebase

Todos os pedidos passam pelo NetworkHandler.enviarPedido, que recebe o endereço, o método (GET, POST, PATCH ou DELETE), o token e o corpo, envia o pedido e devolve um HttpResponse com o código e o corpo. O NetworkHandler não decide se a resposta é boa ou má: isso é trabalho do service. A única exceção é o 401 num pedido com token, que significa sempre o mesmo na aplicação (o token já não é aceite) e lança SessaoExpiradaException. Os pedidos têm 10 segundos de tempo limite e não seguem redirecionamentos, para o token nunca ser enviado para outro endereço. A Listagem 1 mostra o método.

```
 1  public HttpResponse enviarPedido(String endereco, String metodo, String idToken,
 2          String corpo, String contentType) throws IOException {
 3      HttpURLConnection connection = null;
 4      try {
 5          URL url = new URL(endereco);
 6          connection = (HttpURLConnection) url.openConnection();
 7          if (metodo.equals("PATCH")) {
 8              connection.setRequestMethod("POST");
 9              connection.setRequestProperty("X-HTTP-Method-Override", "PATCH");
10          } else {
11              connection.setRequestMethod(metodo);
12          }
13          connection.setConnectTimeout(TIMEOUT_MS);
14          connection.setReadTimeout(TIMEOUT_MS);
15          connection.setInstanceFollowRedirects(false);
16          if (idToken != null) {
17              connection.setRequestProperty("Authorization", "Bearer " + idToken);
18          }
19          if (corpo != null) {
20              connection.setRequestProperty("Content-Type", contentType);
21              connection.setDoOutput(true);
22              try (OutputStream output = connection.getOutputStream()) {
23                  output.write(corpo.getBytes(StandardCharsets.UTF_8));
24              }
25          }
26          int statusCode = connection.getResponseCode();
27          if (statusCode == 401 && idToken != null) {
28              throw new SessaoExpiradaException();
```

```
29          }
30          InputStream stream;
31          if (statusCode >= 200 && statusCode < 300) {
32              stream = connection.getInputStream();
33          } else {
34              stream = connection.getErrorStream();
35          }
36          return new HttpResponse(statusCode, lerResposta(stream));
37      } finally {
38          if (connection != null) {
39              connection.disconnect();
40          }
41      }
42  }
```

**Listagem 1.** Método enviarPedido da classe NetworkHandler

– Linhas 5–6: abre a ligação ao endereço do Firebase.
– Linhas 7–12: o HttpURLConnection não aceita PATCH; o pedido segue como POST com o cabeçalho X-HTTP-Method-Override.
– Linhas 13–15: 10 segundos de tempo limite e sem redirecionamentos.
– Linhas 16–18: o token vai no cabeçalho Authorization, para as regras do Firestore identificarem o aluno.
– Linhas 19–25: se houver corpo (JSON ou formulário), é enviado em UTF-8.
– Linhas 26–29: um 401 num pedido com token lança SessaoExpiradaException.
– Linhas 30–36: lê o corpo da resposta, ou do erro, e devolve-o num HttpResponse com o código.
– Linhas 37–41: a ligação é sempre fechada, mesmo quando há uma exceção.

### 4.2 Do JSON ao modelo

Os dados percorrem sempre o mesmo caminho. Na leitura: JSON → DTO (JsonHandler) → modelo (mapper), e o modelo valida-se. Na escrita, o caminho é o inverso: modelo → DTO (mapper) → JSON (JsonHandler). Por exemplo, para gravar uma amizade, o AmigosService converte o Amigo num AmigoDTO com o AmigoMapper e pede ao JsonHandler o JSON do documento. Só o JsonHandler conhece o formato do Firestore ("fields", "stringValue"); as pesquisas (runQuery) ficam nos services, porque não são dados, dizem ao Firestore o que procurar.

A Listagem 2 mostra o primeiro passo da leitura de uma aula: o documento do Firestore passa a AulaDTO.

```
1  public static AulaDTO json2AulaDTO(JSONObject documento) throws JSONException {
2      JSONObject campos = documento.getJSONObject("fields");
3      AulaDTO dto = new AulaDTO();
4      dto.setNome(campos.getJSONObject("nome").getString("stringValue"));
5      dto.setDiaSemana(Integer.parseInt(
6              campos.getJSONObject("diaSemana").getString("integerValue")));
7      dto.setHoraInicio(campos.getJSONObject("horaInicio").getString("stringValue"));
8      dto.setHoraFim(campos.getJSONObject("horaFim").getString("stringValue"));
9      dto.setSalaId(campos.getJSONObject("salaId").getString("stringValue"));
```

```
10      dto.setTurmaId(campos.getJSONObject("turmaId").getString("stringValue"));
11      return dto;
12  }
```

**Listagem 2.** Método json2AulaDTO da classe JsonHandler

– Linha 2: no Firestore, os dados de um documento estão dentro de "fields".
– Linhas 4–10: cada campo vem com o seu tipo, por exemplo {"stringValue": "DSSMV"}; lê-se o valor e guarda-se no DTO. O dia da semana vem como texto em integerValue e passa a int (linhas 5–6).
– O DTO guarda os dados como vêm da API: as horas continuam em texto ("09:00").

A Listagem 3 mostra o segundo passo: o AulaMapper converte o AulaDTO no modelo.

```
1  public static Aula aulaDTO2Aula(AulaDTO dto) {
2      LocalTime inicio = LocalTime.parse(dto.getHoraInicio());
3      LocalTime fim = LocalTime.parse(dto.getHoraFim());
4      return new Aula(dto.getNome(), dto.getDiaSemana(), inicio, fim,
5              dto.getSalaId());
6  }
```

**Listagem 3.** Método aulaDTO2Aula da classe AulaMapper

– Linhas 2–3: o texto "09:00" passa a LocalTime; uma hora mal escrita lança DateTimeException.
– Linhas 4–5: o construtor da Aula valida os dados (Information Expert) e lança InvalidDataException se forem inválidos, por exemplo uma aula que acaba antes de começar.

### 4.3 Threads, callbacks e Handler

O Android não deixa fazer pedidos de rede na main thread, e só a main thread pode alterar o ecrã. Por isso, cada service corre o pedido numa thread secundária e, no fim, usa um Handler associado à main thread para chamar o callback que o ecrã lhe deu (por exemplo, onResult(aulas, erro)). O ecrã implementa o callback e mostra o resultado ou o erro.

### 4.4 Sessão e sessão expirada

Antes de cada pedido, o AuthService.obterIdTokenValido verifica se o idToken ainda dura mais de 1 minuto; se não, troca o refreshToken por um token novo. Os três casos em que o aluno tem de voltar a autenticar-se (sem sessão em memória, resposta 401 ou renovação recusada) dão todos SessaoExpiradaException, com um só código. Todos os ecrãs reagem da mesma forma, através do MenuConta.sessaoExpirada: mostram a mensagem e voltam ao ecrã de autenticação. A falta de internet não é sessão expirada: continua a ser um erro de ligação.

### 4.5 Lista de amigos

A lista de amigos usa uma ListView com o ListViewAdapterAmigos. O cartão de um amigo está desenhado em XML (item_amigo.xml) e o adapter repete-o por cada amigo, reaproveitando os cartões que saem do ecrã. As fotografias chegam depois da lista: a Activity guarda-as num mapa por uid e pede ao adapter para redesenhar (notifyDataSetChanged).

### 4.6 Deteção de presença na biblioteca

De 30 em 30 segundos, o DetetorPresenca pede ao WifiHelper uma pesquisa de redes e fica com o sinal (RSSI) mais forte dos routers da biblioteca. O método decidir (Listagem 4) usa dois limiares diferentes.

```
 1  private void decidir(Integer rssi) {
 2      if (!ativo) {
 3          return;
 4      }
 5
 6      long agora = SystemClock.elapsedRealtime();
 7      boolean sinalForte = rssi != null && rssi >= BibliotecaConfig.LIMIAR_ENTRADA;
 8      boolean sinalFraco = rssi == null || rssi < BibliotecaConfig.LIMIAR_SAIDA;
 9
10      if (!presente && sinalForte) {
11          marcar(agora);
12      } else if (presente && sinalFraco) {
13          remover();
14      } else if (presente
15              && agora - ultimaRenovacao >= BibliotecaConfig.RENOVAR_PRESENCA_MS) {
16          marcar(agora);
17      } else {
18          agendarProximo();
19      }
20  }
```

**Listagem 4.** Método decidir da classe DetetorPresenca

– Linha 7: o sinal é forte a partir de −65 dBm (LIMIAR_ENTRADA).
– Linha 8: o sinal é fraco abaixo de −75 dBm (LIMIAR_SAIDA), ou se o router não for encontrado.
– Linhas 10–11: quem não estava presente e tem sinal forte é marcado (PATCH a presencas/{uid}).
– Linhas 12–13: quem estava presente e tem sinal fraco é retirado (DELETE).
– Linhas 14–16: quem continua presente renova a hora de 2 em 2 minutos, para não deixar de contar.
– Entre −75 e −65 dBm nada muda: os dois limiares evitam que o aluno "pisque" entre presente e ausente quando está à porta.

Para o aluno contar para a lotação em qualquer ecrã, e não só no ecrã principal, há um só detetor para toda a aplicação. É o padrão Singleton: a classe guarda numa variável static a única instância e só a deixa criar através do método obter (Listagem 5).

```
1  private static DetetorPresenca instancia;
2
3  public static DetetorPresenca obter(Context context) {
```

```
 4      if (instancia == null) {
 5          instancia = new DetetorPresenca(context.getApplicationContext());
 6      }
 7      return instancia;
 8  }
 9
10  private DetetorPresenca(Context context) {
11      this.context = context;
12      this.wifiScanner = new WifiHelper(context);
13  }
```

**Listagem 5.** Singleton da classe DetetorPresenca

– Linha 1: a variável static guarda a única instância, partilhada por todos os ecrãs.
– Linhas 3–8: na primeira chamada cria o detetor; nas seguintes devolve sempre o mesmo. Guarda o Context da aplicação, e não o de um ecrã, porque o detetor vive mais do que cada ecrã.
– Linhas 10–13: o construtor é privado, por isso nenhuma outra classe consegue criar um segundo detetor.

Cada ecrã chama ecraVisivel no onResume e ecraEscondido no onPause. O detetor conta os ecrãs visíveis: liga-se com o primeiro e, quando não resta nenhum (a aplicação foi para fundo), desliga-se passados 2 segundos. A espera existe porque, ao trocar de ecrã, o Android corre o onPause do ecrã antigo antes do onResume do novo.

# 5 Testes

Há testes unitários, que verificam uma classe de cada vez, e testes funcionais, que verificam a aplicação inteira a funcionar.

## 5.1 Testes unitários

Os testes unitários usam JUnit e estão no projeto, em app/src/test (correm no computador) e em app/src/androidTest (correm no emulador ou no telemóvel). A Tabela 12 resume o que cada classe de teste verifica. São 43 testes, e todos passam.

**Tabela 12.** Testes unitários

| Classe de teste | Testes | Onde corre | O que verifica |
|---|---|---|---|
| NetworkHandlerTest | 7 | Computador | Os pedidos GET e POST, o cabeçalho com o token, o UTF-8, as respostas 201, 204 e 403, os redirecionamentos e o 401 (sessão expirada só quando há token). Usa um servidor HTTP local (127.0.0.1): não contacta o Firebase. |
| AulaTest | 6 | Computador | Uma aula válida é criada; disciplina vazia, dia fora de 1 a 7, fim antes do início e falta de sala são recusados; o estaADecorrer. |
| AmigoTest | 6 | Computador | Um amigo válido é criado; uid ou nome vazios, número sem 7 dígitos e posição fora dos limites são recusados; o temPosicao. |

| Classe de teste | Testes | Onde corre | O que verifica |
|---|---|---|---|
| LocalTest | 4 | Computador | Um local válido é criado; id ou nome vazios e coordenadas fora dos limites são recusados. |
| AulaMapperTest | 2 | Computador | As horas em texto ("09:00") passam a LocalTime; uma hora mal escrita dá erro. |
| AmigoMapperTest | 3 | Computador | AmigoDTO → Amigo, AlunoDTO → Amigo e Amigo → AmigoDTO, com os mesmos dados. |
| LocalizacaoAmigoMapperTest | 1 | Computador | A posição do LocalizacaoDTO fica guardada no Amigo. |
| JsonHandlerTest | 14 | Emulador ou telemóvel | A leitura de cada tipo de JSON do Firebase (aula, aluno, amigo, local, posição, sessão, contagem e erro) e a escrita do JSON de cada pedido (login, amigo, foto, posição e presença). Corre no Android porque usa o org.json. |

## 5.2 Testes funcionais

Os testes funcionais da Tabela 13 verificam a aplicação inteira, do ecrã ao Firebase. Foram feitos à mão, num telemóvel Android e no emulador, seguindo o cenário de cada linha e comparando com o resultado esperado.

**Tabela 13.** Testes funcionais

| ID | Cenário | Resultado esperado |
|---|---|---|
| TF01 | Autenticação — número com 6 dígitos | "O número de estudante deve ter exatamente 7 dígitos."; não é feito nenhum pedido. |
| TF02 | Autenticação — password com 11 caracteres | "A password não pode exceder 10 caracteres." |
| TF03 | Autenticação — credenciais válidas | Abre o ecrã principal. |
| TF04 | Autenticação — password errada | "Utilizador ou password incorretos."; o formulário volta a ficar ativo. |
| TF05 | Autenticação — sem Internet | Mensagem de erro de ligação. |
| TF06 | Sair e voltar em menos de 2 minutos | Entra no ecrã principal sem nova autenticação. |
| TF07 | Sair e voltar após mais de 2 minutos | Volta ao ecrã de autenticação. |
| TF08 | UC01 — dia com aulas | Aulas do dia por ordem, com nome, horas e sala. |
| TF09 | UC01 — dia sem aulas | "Não tens aulas marcadas para hoje." |
| TF10 | UC02 — primeira abertura do mapa | Pede autorização; depois de aceitar, aparece a fotografia do aluno (contorno azul). |
| TF11 | UC02 — localização recusada | O mapa aparece sem a posição do aluno. |
| TF12 | UC02 — dia com aulas | Aparecem a biblioteca e a sala da aula atual ou da próxima. |
| TF13 | UC02 — amigo com localização partilhada | A fotografia do amigo (contorno vermelho) acompanha a posição dele. |

| ID | Cenário | Resultado esperado |
|---|---|---|
| TF14 | UC02 — desligar a localização com o mapa aberto (Android 10) | A aplicação continua aberta. |
| TF15 | UC03 — sem localização precisa | O cartão mostra o botão "Ativar localização". |
| TF16 | UC03 — com localização precisa | "N pessoas neste momento" (ou "1 pessoa"). |
| TF17 | UC03 — aluno junto do router da biblioteca | A lotação aumenta 1 em menos de 30 s; ao afastar-se, diminui. |
| TF18 | UC04 — abrir Amigos | Cada amigo com fotografia, nome e número. |
| TF19 | UC05 — Ver horário de um amigo | Aulas de hoje do amigo, ou "Este amigo não tem aulas hoje." |
| TF20 | UC06 — número de outro aluno | "Amigo adicionado."; o amigo aparece nas duas listas. |
| TF21 | UC06 — número inexistente, o próprio ou amigo repetido | A mensagem certa para cada caso; nada é gravado. |
| TF22 | UC07 — remover um amigo | "Amigo removido."; desaparece das duas listas. |
| TF23 | UC08 — escolher uma imagem | A fotografia aparece no perfil e na lista de amigos dos outros. |
| TF24 | Menu ≡ — Terminar sessão | A sessão é apagada e abre o ecrã de autenticação. |
| TF25 | UC02 — amigo que não envia a posição há mais de 5 minutos | O amigo continua no mapa, com a fotografia a cinzento e "visto há …" por cima. |
| TF26 | UC03 — aluno junto do router com o ecrã Amigos aberto; depois, aplicação em fundo | A presença é marcada em menos de 30 s, sem voltar ao ecrã principal; em fundo, deixa de ser renovada. |

A Fig. 31 mostra três destes testes no emulador. Em (a), o ecrã de autenticação, antes do TF03. Em (b), a página Início depois de entrar, num dia sem aulas (TF09): o horário mostra "Não tens aulas marcadas para hoje.". Em (c), a página Mapa com a fotografia de um amigo no campus (TF13).

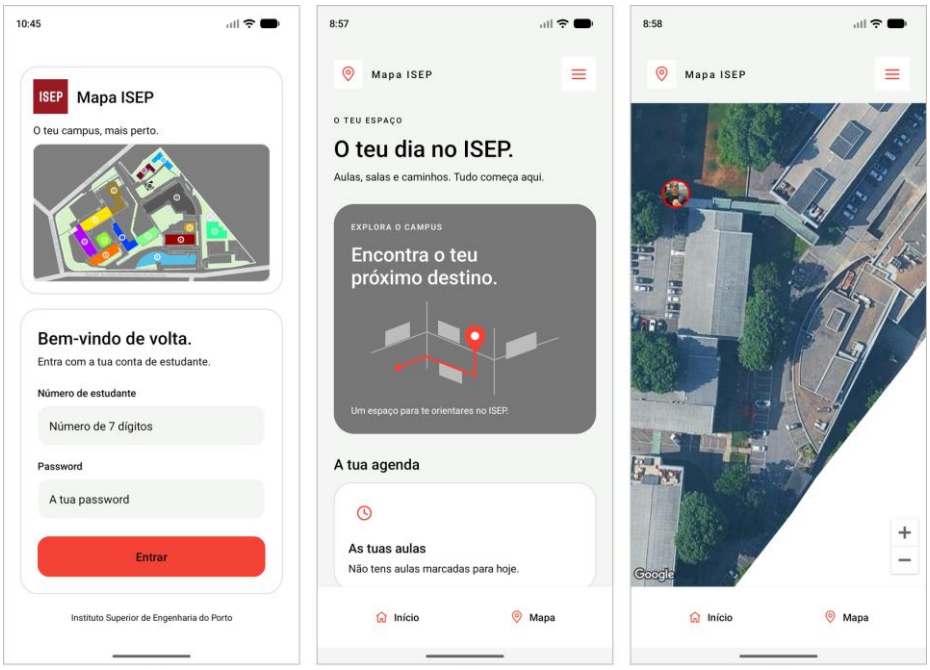



**Fig. 31.** Aplicação: (a) autenticação (TF03), (b) Início sem aulas (TF09), (c) mapa com um amigo (TF13)

A Fig. 32 mostra mais três. Em (a), a tentativa de adicionar um aluno que já é amigo: o campo mostra "Este aluno já é teu amigo." e nada é gravado (TF21). Em (b), o horário de hoje desse amigo, aberto com o botão Ver horário (TF19). Em (c), o perfil com a fotografia escolhida pelo aluno (TF23).

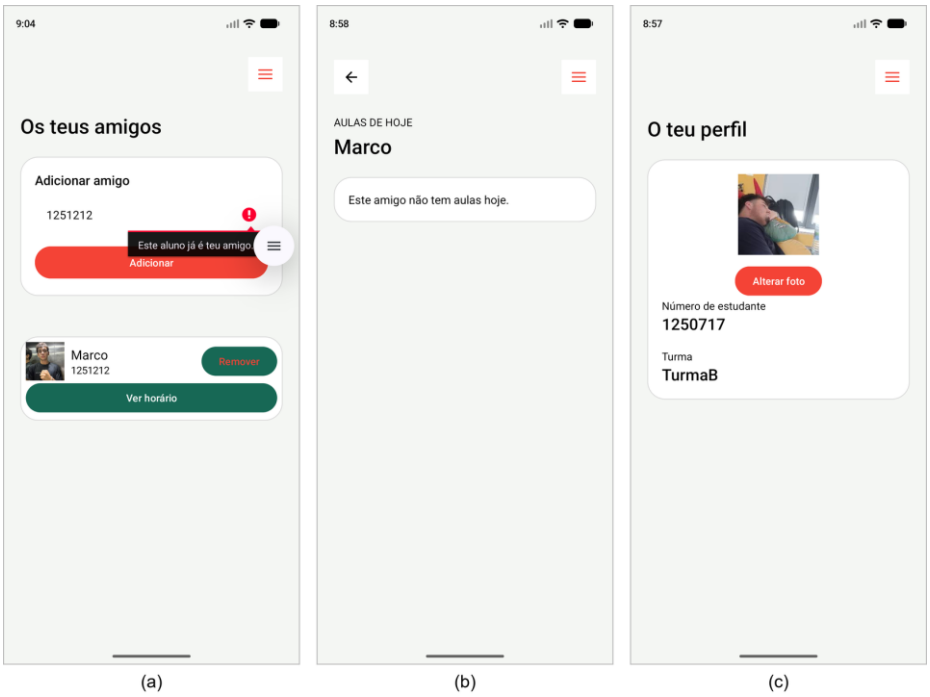



**Fig. 32.** Aplicação: (a) amigo repetido (TF21), (b) horário de um amigo (TF19), (c) perfil (TF23)

# 6 Conclusão

Esta secção faz o balanço do projeto: o que ficou a funcionar, as limitações que se conhecem e as principais dificuldades encontradas.

## 6.1 O que funciona e o que não funciona

Os oito casos de uso estão implementados e passaram nos testes funcionais da Tabela 13: o horário do dia, o mapa com a sala da próxima aula e as posições do aluno e dos amigos, a estimativa de ocupação da biblioteca, a lista de amigos com fotografia, o horário de um amigo, adicionar e remover amigos e a fotografia de perfil. A autenticação, a renovação do token e a sessão expirada funcionam em todos os ecrãs.

Há limitações que vêm da forma como a aplicação funciona. A estimativa da biblioteca só conta os alunos que têm a aplicação aberta com a localização precisa ativa, e os limiares de sinal foram medidos dentro da biblioteca. A posição de um amigo só é atualizada enquanto ele tem a página Mapa aberta; passados 5 minutos sem atualização, aparece a cinzento com a indicação de há quanto tempo foi visto.

### 6.2 Dificuldades

– **REST à mão.** Sem o SDK do Firebase, foi preciso construir os pedidos, os cabeçalhos e o JSON, incluindo o PATCH, que o HttpURLConnection não aceita.
– **Threads.** A comunicação teve de sair da main thread, com threads, callbacks e um Handler para devolver o resultado ao ecrã.
– **Wi-Fi.** O Android limita as pesquisas de redes (no máximo 4 em 2 minutos) e só as permite com a localização precisa; por isso o detetor usa 30 segundos e o cartão da biblioteca pede essa permissão [4].
– **Organização.** O ecrã principal chegou a ter perto de 800 linhas. Foi dividido em partes com um só trabalho, e o código foi reorganizado segundo o exemplo da unidade curricular.

## Referências


1. Sousa, P.B.: DSSMV – M1: ProjectDroid, Mobile Android Project. ISEP, Porto (2026)
2. Sousa, P.B., Freitas, C.: DSSMV – School Disk Client, Technical Report (TP). ISEP, Porto (2026)
3. Android Developers: LocationListener, https://developer.android.com/reference/android/location/LocationListener
4. Android Developers: Wi-Fi scanning overview, https://developer.android.com/develop/connectivity/wifi/wifi-scan
5. Firebase: Firebase Auth REST API, https://firebase.google.com/docs/reference/rest/auth
6. Firebase: Cloud Firestore REST API, https://firebase.google.com/docs/firestore/reference/rest
7. Google: Maps SDK for Android, https://developers.google.com/maps/documentation/android-sdk
8. Larman, C.: Applying UML and Patterns, 3rd edn. Prentice Hall, Upper Saddle River (2004)